Subject:	OOP-JAVA
Name:	Yug Gandhi
ID No.:	25AIML013

Practical-12

GIT REPOSITRY LINK : https://github.com/25aiml013-sudo/JAVA-PRACTICALS.git

Part A

PROBLEM-1 :

Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

CODE :

```
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {
        String text = "java is easy java is powerful java";
        Map<String, Integer> map = new HashMap<>();

        for (String word : text.split(" ")) {
            map.put(word, map.getOrDefault(word, 0) + 1);
        }

        List<Map.Entry<String, Integer>> list =
            new ArrayList<>(map.entrySet());
```

```
        list.sort((a, b) -> b.getValue().compareTo(a.getValue()));

        for (Map.Entry<String, Integer> e : list) {
            System.out.println(e.getKey() + " : " + e.getValue());
        }
    }
}
```

OUTPUT :

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users/yuggand
 java WordFrequency
java : 3
is : 2
powerful : 1
easy : 1
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

PROBLEM-2 :

Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

CODE :

```
import java.util.*;

public class UniqueVisitors {
    public static void main(String[] args) {
        List<String> users = Arrays.asList(
```

```
            "Rahul", "Amit", "Priya", "Rahul", "Neha", "Amit"
        );

        HashSet<String> unique = new HashSet<>(users);
        LinkedHashSet<String> ordered = new LinkedHashSet<>(users);

        System.out.println("Unique users: " + unique.size());

        int count = 0;
        for (String name : ordered) {
            if (count == 3) break;
            System.out.println(name);
            count++;
        }
    }
}
```

OUTPUT :

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users/yuggandhi/Desktop/S
& java UniqueVisitors
Unique users: 4
Rahul
Amit
Priya
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

PROBLEM-3 :

Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

CODE :

```
import java.util.*;

public class Leaderboard {
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();

        scores.put("Rahul", 90);
        scores.put("Amit", 85);
        scores.put("Priya", 95);
        scores.put("Neha", 90);

        List<Map.Entry<String, Integer>> list =
            new ArrayList<>(scores.entrySet());

        list.sort((a, b) -> {
            int result = b.getValue().compareTo(a.getValue());
            if (result == 0)
                return a.getKey().compareTo(b.getKey());
            return result;
        });

        int topN = 3;
        for (int i = 0; i < Math.min(topN, list.size()); i++) {
            System.out.println(
                list.get(i).getKey() + " : " +
                list.get(i).getValue()
            );
```

```
    }
  }
}
```

OUTPUT :

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users
ava Leaderboard
Priya : 95
Neha : 90
Rahul : 90
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

Key Questions :

1. Difference between List, Set, and Map:

- List: Stores ordered elements and allows duplicates.
- Set: Stores unique elements without duplicates.
- Map: Stores data as key-value pairs with unique keys.

2. When do you use HashMap versus TreeMap?

- HashMap: Use when fast data access is needed without sorted keys.
- TreeMap: Use when keys must be stored in sorted order.

3. Difference between Comparable and Comparator:

- Comparable: Defines the natural sorting order using the compareTo() method inside the class.
- Comparator: Defines custom sorting using the compare() method, usually in a separate class or lambda expression.